Class 34 Exploring Wireshark For Network Analysis

Network Traffic

Definition: Network traffic is the data transmitted between devices over a network when performing activities such as browsing websites, sending emails, or streaming videos.

How Network Traffic Works

1. Data splitting: The operating system divides data into smaller units for transmission.
2. Encapsulation: Each network layer adds its own control information, such as addressing and delivery details.
3. Transmission: The data travels across the network using protocols such as TCP, UDP, and IP.
4. Reassembly: The receiving device processes the data and reconstructs the original information when required.

Main Components of a Network Packet

- Header: Contains control information required for communication, such as source and destination IP addresses, port numbers, flags, sequence numbers, and checksums, depending on the protocol.
- Payload: Contains the actual data being carried, such as website content, image data, or encrypted application data.

Example

When you visit a website:

- Header: Contains information about where the data comes from and where it needs to go.
- Payload: Contains website content, such as HTML, or encrypted data when HTTPS is used.

Key Terms

- Packet: A unit of data transmitted across a network.

- Encapsulation: The process of adding protocol-specific headers, and sometimes trailers, to data.
- OSI Model: A seven-layer model used to understand network communication.
- TCP/IP Model: A practical networking model used for internet communication.
- Port Number: Identifies a particular service or application endpoint, such as HTTPS commonly using port `443`.

Remember: Network traffic analysis helps security analysts identify suspicious connections, malicious downloads, command-and-control communication, and unusual data transfers.

Wireshark Basics & Architecture

Definition: Wireshark is a free, open-source network protocol analyzer used to capture, inspect, and analyze network traffic in real time or from previously saved capture files.

Main Functions

- Packet Capture: Collects network packets from a selected network interface.
- Packet Inspection: Displays packet headers, protocol details, and available payload data.
- Protocol Analysis: Helps investigate protocols such as TCP, UDP, DNS, HTTP, and TLS.
- Offline Analysis: Opens previously captured traffic files, such as `.pcap` and `.pcapng`.

How Wireshark Works

1. Capture: Collects packets from a network interface using a capture mechanism such as libpcap or Npcap.
2. Decode: Identifies protocols and interprets packet fields.
3. Display: Shows packets in a list with detailed protocol information.
4. Analyze: Allows users to apply filters and investigate suspicious traffic.

Wireshark Interface

- Packet List: Displays captured packets with columns such as source, destination, protocol, and length.
- Packet Details: Shows the selected packet's protocol layers and header fields.
- Packet Bytes: Displays the packet's raw data in hexadecimal and ASCII where possible.
- Display Filter Bar: Filters displayed packets without deleting the captured data.

Important Terms

- Packet Sniffer: A tool that captures and examines network packets.
- `.pcap`: A common file format for storing packet captures.
- `.pcapng`: A newer, more flexible packet-capture file format.
- Capture Filter: Limits which packets are collected during capture.
- Display Filter: Limits which captured packets are shown for analysis.

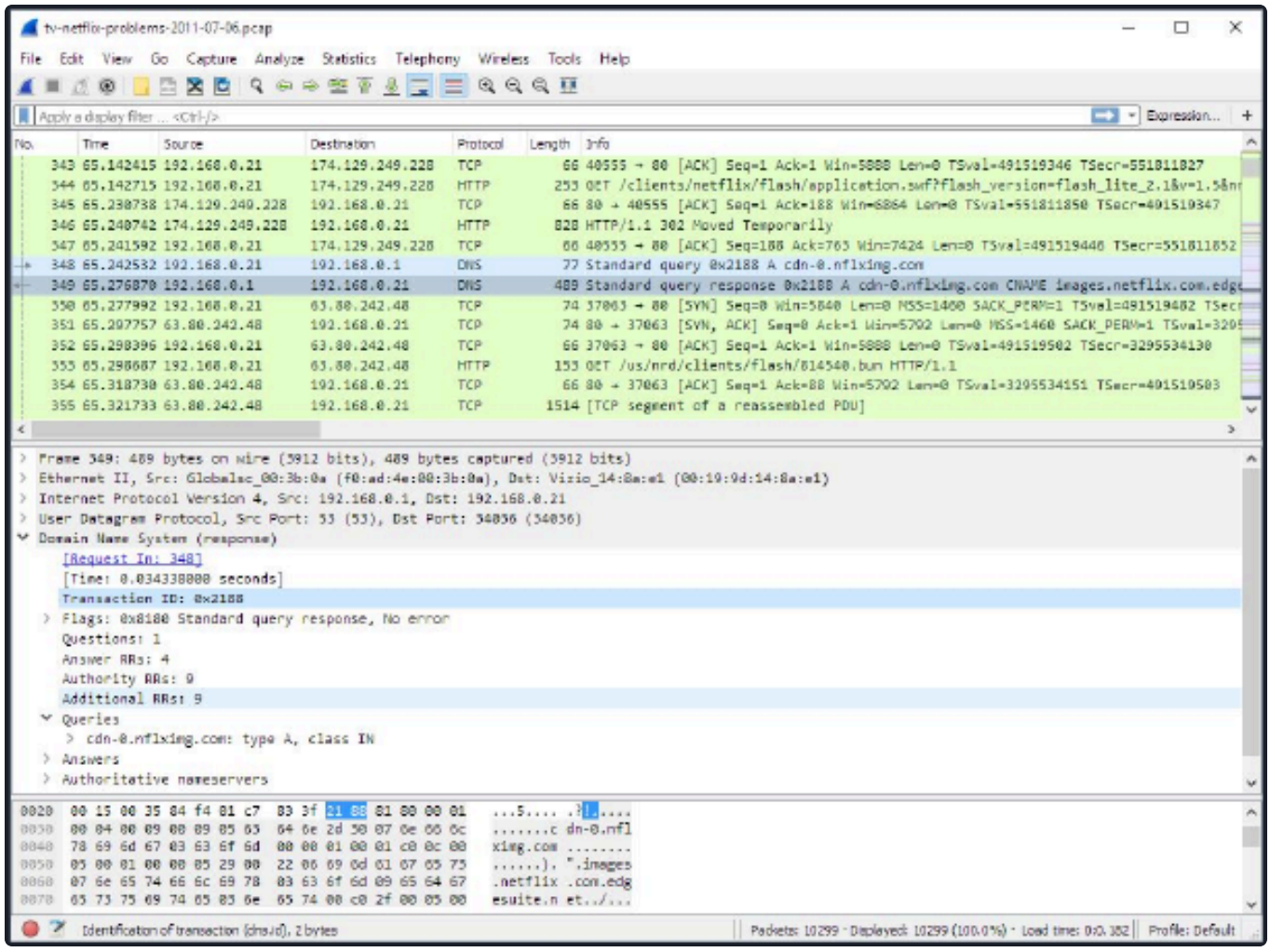
Example

To investigate a suspicious DNS request, an analyst can use the display filter:

```
dns
```

Wireshark then displays packets identified as DNS traffic, helping the analyst inspect requested domains and DNS responses.

Remember: Wireshark helps security analysts investigate suspicious connections, malware communication, unusual traffic patterns, and potential data exfiltration. It analyzes network traffic; it does not automatically prove that a packet or connection is malicious.



How Wireshark Captures Packets

Wireshark captures network packets using the network interface, packet-capture libraries, and protocol dissectors to make network traffic understandable.

1. Promiscuous Mode

Definition: A mode in which a network interface card (NIC) can pass received frames to the operating system even when their destination MAC address is not the NIC's own address.

- Normal mode: The NIC generally accepts frames addressed to it, along with certain broadcast or multicast traffic.
- Promiscuous mode: Allows the NIC to pass more received frames to the operating system for analysis.
- Monitor mode (Wi-Fi): Allows compatible wireless adapters to capture 802.11 frames over the air, including certain frames not addressed to the capturing device.

Important: Promiscuous mode does not automatically let Wireshark see all network traffic. Switches, Wi-Fi encryption, adapter capabilities, and network configuration affect what can be captured.

2. Capture Core Libraries

Wireshark relies on packet-capture libraries to communicate with the operating system's networking facilities and drivers.

- libpcap: Commonly used on Linux and other Unix-like systems.
- Npcap: Commonly used on Windows; it is the modern packet-capture platform associated with Wireshark.

Purpose: These libraries provide Wireshark with access to captured packets.

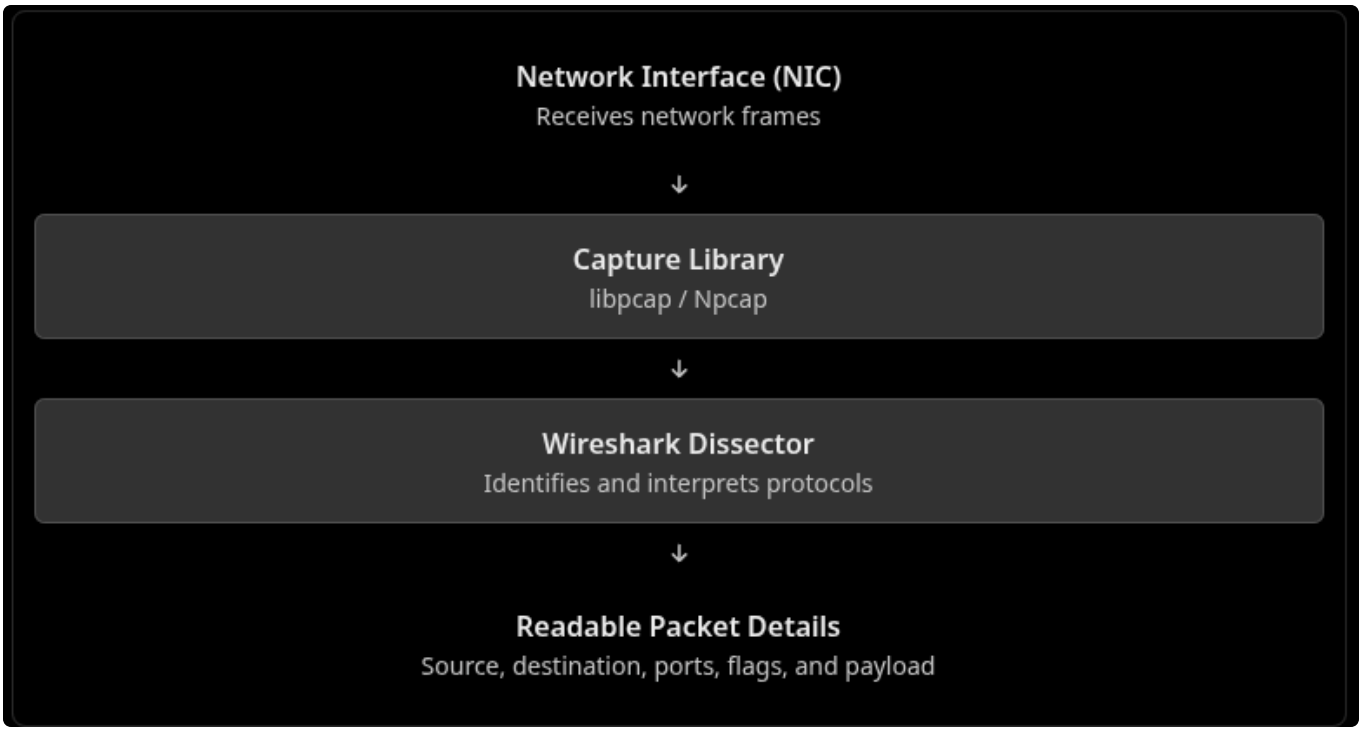
3. Dissectors

Definition: Dissector modules interpret packet bytes according to known network protocols and display the results in a readable format.

Examples:

- DNS dissector: Interprets domain-name queries and responses.
- TCP dissector: Displays ports, sequence numbers, flags, and other TCP fields.
- HTTP dissector: Interprets HTTP requests and responses when the contents are available.
- ARP dissector: Interprets IPv4 address-to-MAC address resolution messages.

How the Complete Process Works



Quick Revision

Component	Main purpose
Promiscuous mode	Passes more received frames to the OS
Monitor mode	Captures wireless frames using a compatible Wi-Fi adapter
libpcap	Packet capture on Unix-like systems
Npcap	Packet capture on Windows
Dissectors	Decode packet bytes into protocol fields

Remember: NIC receives frames → capture library provides packets → dissectors decode protocols → Wireshark displays readable details.

Wireshark Packet Operations

Wireshark packet operations help analysts filter network traffic, inspect communication between devices, and interpret packets during security investigations.

1. Capture Filters vs. Display Filters

Wireshark has two types of filters.

Capture Filters (BPF Syntax)

Definition: Capture filters specify which packets Wireshark collects before or during packet capture.

- Discard unwanted packets before they are saved in the capture.
- Can reduce the amount of data captured and stored.
- Use Berkeley Packet Filter (BPF) syntax.

Example:

```
host 192.168.1.1 and port 80
```

Captures traffic to or from `192.168.1.1` involving port `80`.

Display Filters (Wireshark Syntax)

Definition: Display filters specify which already captured packets are shown on the screen.

- Hide irrelevant packets without deleting them from the capture file.
- Useful for investigating specific IP addresses, protocols, and ports.
- Use Wireshark's own filter syntax.

Example:

```
ip.addr == 192.168.1.1 && tcp.port == 80
```

Shows packets involving IP address `192.168.1.1` and TCP port `80`.

Key Differences

Feature	Capture Filter	Display Filter
When applied	Before/during capture	After capture, while viewing
Purpose	Controls which packets are collected	Controls which packets are displayed
Syntax	BPF	Wireshark filter syntax

Feature	Capture Filter	Display Filter
Effect	Unmatched packets are not captured	Unmatched packets remain available
Example	`port 53`	`udp.port == 53`

2. Crucial Display Filter Cheat Sheet

Filter by IP Address

```
ip.src == 10.0.0.5
```

Shows packets with source IP `10.0.0.5`.

```
ip.dst == 10.0.0.1
```

Shows packets with destination IP `10.0.0.1`.

```
ip.addr == 192.168.1.1
```

Shows packets where the IP is either the source or destination.

Filter by Protocol

```
dns
http
arp
icmp
tls
```

Displays packets identified as DNS, HTTP, ARP, ICMP, or TLS traffic, respectively.

Filter by Port

```
tcp.port == 443
```

Shows TCP packets involving port `443`, commonly used for HTTPS.

```
udp.port == 53
```

Shows UDP packets involving port `53`, commonly used for DNS.

Boolean Operators

- `&&` — AND: both conditions must be true.
- `||` — OR: at least one condition must be true.
- `!` — NOT: reverses a condition.

Example:

```
ip.addr == 192.168.1.1 && !tcp.port == 80
```

Shows packets involving `192.168.1.1` that do not involve TCP port `80`.

Filter by Application Fields

```
http.request.method == "POST"
```

Shows HTTP requests using the POST method.

```
dns.qry.name contains "example"
```

Shows DNS queries whose queried name contains `example`.

3. Decoding and Standard Operations

Follow Stream

Definition: Reconstructs the application-level conversation associated with a particular network stream.

- Right-click a packet and select Follow → TCP Stream.
- For supported UDP traffic, use the corresponding UDP stream option.
- Helps examine requests, responses, and communication in sequence.

Example: Investigating an HTTP connection to understand what the client requested and what the server returned.

Note: Encrypted TLS traffic generally cannot be read as plaintext unless the necessary decryption information is available.

Decode As

Definition: Manually tells Wireshark to interpret traffic using a selected protocol when automatic protocol identification does not work.

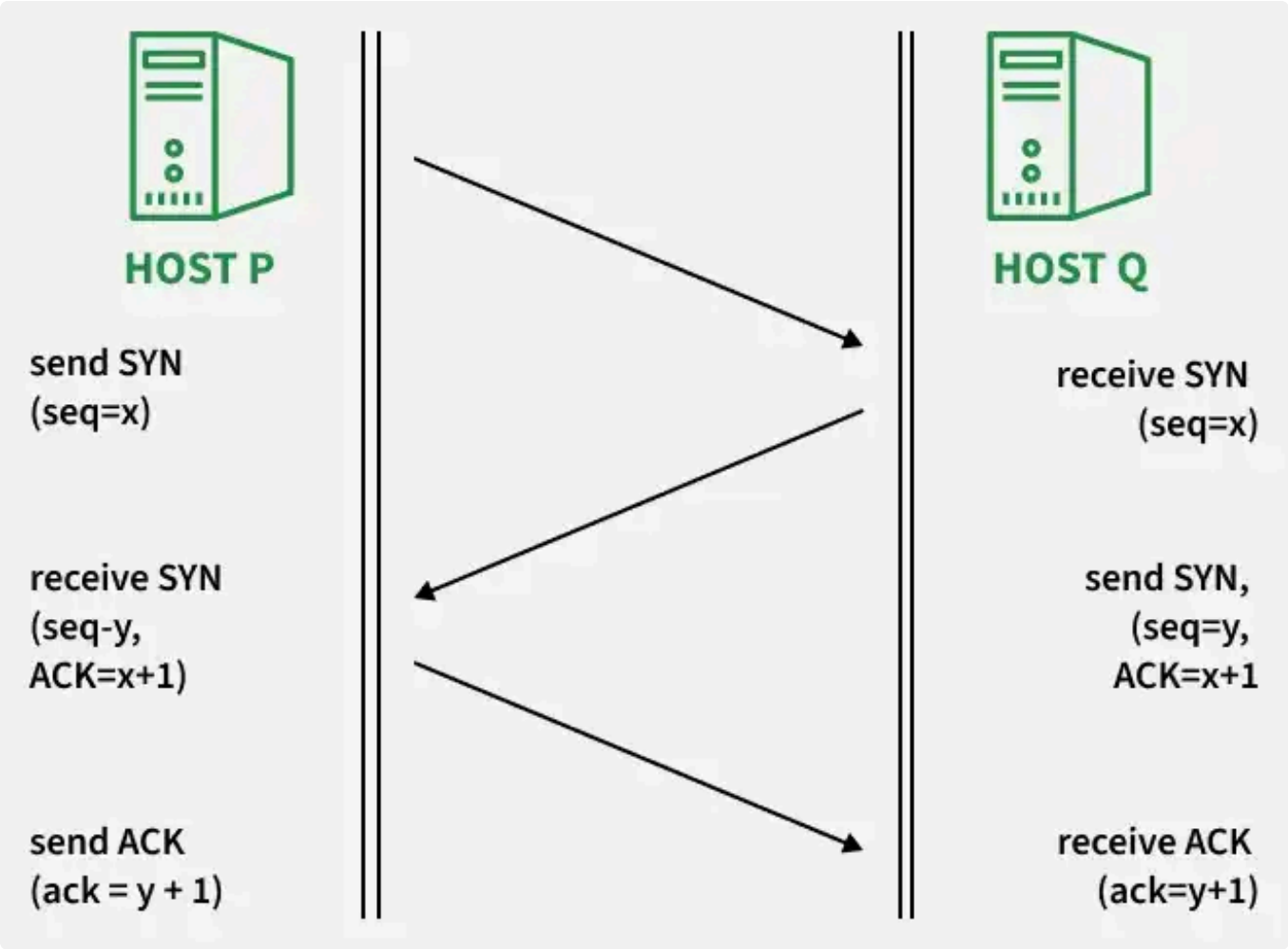
Example: HTTP traffic running on a non-standard port, such as `8080`, may not be identified as HTTP automatically. You can use Decode As to assign the appropriate dissector.

Quick Revision

Operation	Purpose
Capture filter	Select packets to collect
Display filter	Select packets to show
Follow Stream	Reconstruct a conversation
Decode As	Manually assign a protocol dissector
`ip.addr`	Match source or destination IP
`tcp.port`	Match TCP source or destination port
`contains`	Search for a substring in a supported field

SOC Analyst tip: During an investigation, start by filtering on the suspicious IP, domain, protocol, or port. Then follow relevant streams and inspect packet details to determine whether the communication is legitimate or potentially malicious.

Stnadard TCP 3 Way Handshake



- ****Step 1 (SYN):** In the first step, the client wants to establish a connection with a server, so it sends a segment with SYN(Synchronize Sequence Number) which informs the server that the client is likely to start communication and with what sequence number it starts segments with
- ****Step 2 (SYN + ACK):** Server responds to the client request with SYN-ACK signal bits set. Acknowledgement(ACK) signifies the response of the segment it received and SYN signifies with what sequence number it is likely to start the segments with
- ****Step 3 (ACK):** In the final part client acknowledges the response of the server and they both establish a reliable connection with which they will start the actual data transfer.

Flag	**Purpose**
****SYN (Synchronize)**	Initiates a TCP connection by sending the client's Initial Sequence Number (ISN).

Flag	**Purpose**
****ACK (Acknowledgment)****	Acknowledges the received sequence number and confirms successful receipt of the segment.
****SYN + ACK (Combination of SYN and ACK Flags)****	Acknowledges the client's ISN while sending the server's own Initial Sequence Number (ISN).

Step 1: Client -> Server (SYN)

- Sequence Number = 1000
- SYN flag = 1
- No acknowledgment number (since this is the first segment)

The client proposes its starting sequence value.

Step 2: Server -> Client (SYN-ACK)

- Sequence Number = 5000
- Acknowledgment Number = 1001
- (1000 + 1)

The server acknowledges the client’s SYN by adding 1 to the client’s sequence number.
The value increases by 1 because the SYN consumes one sequence number.

Step 3: Client -> Server (ACK)

- Sequence Number = 1001
- Acknowledgment Number = 5001
- (5000 + 1)

The client acknowledges the server’s SYN by adding 1 to the server’s sequence number.

How Synchronization Happens

- Each side shares its Initial Sequence Number.
- Each side confirms the other’s sequence number using acknowledgment.
- After both acknowledgments are verified, the connection enters the ESTABLISHED state.

This ensures both endpoints agree on starting sequence values before actual data transmission begins.

Wireshark: Performance Issues & Security Analysis

1. Identifying Network Performance Issues

Wireshark uses TCP Expert Info to highlight events that may indicate network delays, packet loss, or communication problems.

A. TCP Retransmissions

- Meaning: TCP retransmits data when it believes previously sent data has not been successfully acknowledged.
- Cause: Packet loss, network congestion, or delayed acknowledgements.
- Impact: Can increase latency and slow data transfer.

Example: A sender transmits data but does not receive the expected ACK in time, so it sends the data again.

B. TCP Duplicate ACKs

- Meaning: The receiver repeatedly acknowledges the last in-order data it has received.
- Cause: A segment may be missing, or packets may have arrived out of order.
- Impact: Can indicate packet loss or reordering and may trigger TCP fast retransmission.

C. TCP Zero Window

- Meaning: The receiver advertises a TCP receive window of zero, telling the sender to stop sending additional data temporarily.
- Cause: The receiving application may not be processing incoming data quickly enough, causing its receive buffer to fill.
- Impact: Data transfer pauses or slows until the receiver advertises available buffer space.

Quick Comparison

TCP event	Possible indication
Retransmission	Lost or unacknowledged data
Duplicate ACK	Missing or out-of-order segment
Zero Window	Receiver cannot currently accept more data

Important: These events are clues, not proof of a fault. Investigate surrounding packets and both endpoints before deciding the cause.

2. Security Analysis

A. Detecting Network Scanning

Definition: Network scanning is the process of probing hosts, ports, or services to discover which systems and services are available.

Tools such as Nmap can generate recognizable traffic patterns.

- SYN scan: Many TCP SYN packets probe ports without completing normal TCP connections.
- SYN-ACK followed by RST: A response pattern often seen when a SYN scan finds an open port but does not complete the handshake.
- Xmas scan: Uses unusual combinations of TCP flags, typically FIN, PSH, and URG.
- Null scan: Sends TCP packets with no flags set.

How to investigate: Examine the source IP, destination ports, packet frequency, TCP flags, and responses.

Remember: Legitimate applications and network tools can also produce unusual traffic. Confirm the pattern before labelling it malicious.

3. TLS Decryption in Wireshark

Definition: TLS (Transport Layer Security) encrypts network communication to protect data from unauthorized reading.

Wireshark normally displays TLS packet metadata, but the encrypted application payload cannot generally be read without appropriate decryption secrets.

How TLS Decryption Works

1. A supported browser creates TLS session keys.
2. The browser can be configured to write session secrets to a file using the `SSLKEYLOGFILE` environment variable.
3. Wireshark is configured to use that key-log file.
4. If the secrets match the captured TLS session, Wireshark can decrypt supported traffic and display the application data.

Important terms:

- TLS: Protocol that protects network communications through encryption and authentication.
- Session keys: Cryptographic keys used to protect data during a TLS session.
- SSLKEYLOGFILE: An environment variable commonly used to specify where supported applications write TLS session secrets.
- HTTPS: HTTP communication protected by TLS.

Security note: Session-key logs are sensitive because they can allow captured sessions to be decrypted. Use them only for authorized analysis and protect the files carefully.

Quick Revision

- Retransmission: Data is sent again because acknowledgement or delivery is uncertain.
- Duplicate ACK: May indicate a missing or out-of-order segment.
- Zero Window: Receiver temporarily cannot accept more data.
- SYN scan: Probes TCP ports using SYN packets.
- Xmas / Null scan: Uses unusual TCP flag patterns.
- TLS decryption: Requires suitable session secrets for the captured traffic.

SOC Analyst workflow: Detect the unusual pattern → identify the source and destination → inspect surrounding packets → correlate with endpoint or firewall logs → document findings and escalate if suspicious.